GNCore: practical results explained

What this tells you

This is a practical pilot of GNCore and a published reference for each of its 30 developer categories. Original Microsoft VS Code and Antigravity were tested separately. The charts show real saved answers, executed code tests and browser observations; a missing measurement stays gray. Development and holdout tasks are different and must not be subtracted to claim an improvement.

How to read a percentage

A result of 5/6 (83%) means five of six specific checks passed on one task. It does not mean the skill is 83% intelligent, an application is 83% secure, or that future tasks have an 83% success probability. The 95% line is an improvement target, never a value filled into the data.

Results currently supported

All 30 categories have saved answers in both IDEs. The main comparison uses the same holdout tasks for original GNCore, published references and the revised trial. Twenty-nine categories are fully reviewed; the web interface category still needs rendered checks. The four landscape sheets show every category, raw counts and percentages.

Development tasks were used to find weaknesses; holdout tasks were reserved for the next evaluation. The final three-case follow-up uses already known tasks and is reported separately.

Did the revised trial do better?

On the same holdout tasks, the 29 fully reviewed non-UI categories passed164/173 checks (94.8%) before and168/173 (97.1%) in the revised VS Code trial; the published references passed158/173 (91.3%). Antigravity passed140/173 (80.9%) before and147/173 (85.0%) in the revised trial; references passed132/173 (76.3%). These are single unblinded pilot observations, not statistically established gains or a universal ranking. Many individual categories remain below95%, and the initial revised trial is not the final three-entrypoint follow-up.

What changed

All 30 GNCore entrypoints gained original, domain-specific failure-prevention guidance. The work addresses recoverable retries, safe migration boundaries, model/data correctness, reproducible builds and visible UI focus. Existing workflows and upstream license notices remain intact. A text change is an implemented improvement to guidance; only a comparable held-out result can support an outcome-improvement claim.

A passing test suite still missed a bug

VS Code's initial revised allocation code passed all16 frozen cases but failed three added mixed-remainder cases. The original answer and failures are retained. These exploratory tests were added after inspecting the bug and are reported separately, not disguised as predeclared coverage. This is why100% on a small fixture cannot certify general correctness.

antigravity-gncore-after-holdout: 3/3 added allocation cases passed.

antigravity-gncore-before-holdout: 3/3 added allocation cases passed.

antigravity-gncore-final-followup-holdout: 3/3 added allocation cases passed.

antigravity-reference-holdout: 3/3 added allocation cases passed.

vscode-gncore-after-holdout: 0/3 added allocation cases passed.

vscode-gncore-before-holdout: 3/3 added allocation cases passed.

vscode-gncore-final-followup-holdout: 3/3 added allocation cases passed.

vscode-reference-holdout: 3/3 added allocation cases passed.

What remains uncertain

There is one task variant per category and no repeated-trial estimate. Manual scoring is unblinded. The whole-library batch can create cross-task context and retrieval effects. Copilot Auto can choose models; these results cannot isolate a skill's causal effect from model knowledge. Final code-build, performance and frontend guidance changed after the first revised trial and receive a separate adaptive follow-up. The frontend style section was restored after a generation regression. These known-task follow-ups are not spliced into the original holdout scores. Browser use became disabled after the development tests; missing holdout UI evidence remains gray. This is not a broad ranking of publishers or a security certification. Visual beauty was not assigned a fabricated percentage.

Final guidance: separate follow-up

Antigravity: 16/16 frozen allocation tests passed.

Antigravity: 3/3 exploratory allocation tests passed.

Antigravity: arithmetic is correct. The performance answer still omits throughput coverage. Browser behavior remains unmeasured.

VS Code: 16/16 frozen allocation tests passed.

VS Code: 3/3 exploratory allocation tests passed.

VS Code: arithmetic is correct. The performance answer still omits throughput coverage. Browser behavior remains unmeasured.

Both final allocation answers pass the added regressions, and both arithmetic answers give22.4 and28ms correctly. A known-task follow-up cannot establish generalization. The original VS Code baseline also exposed an extreme-logit neural-network failure (13/14 tests); initial revised and reference answers passed14/14 on that fixture. Earlier failures remain in the evidence.

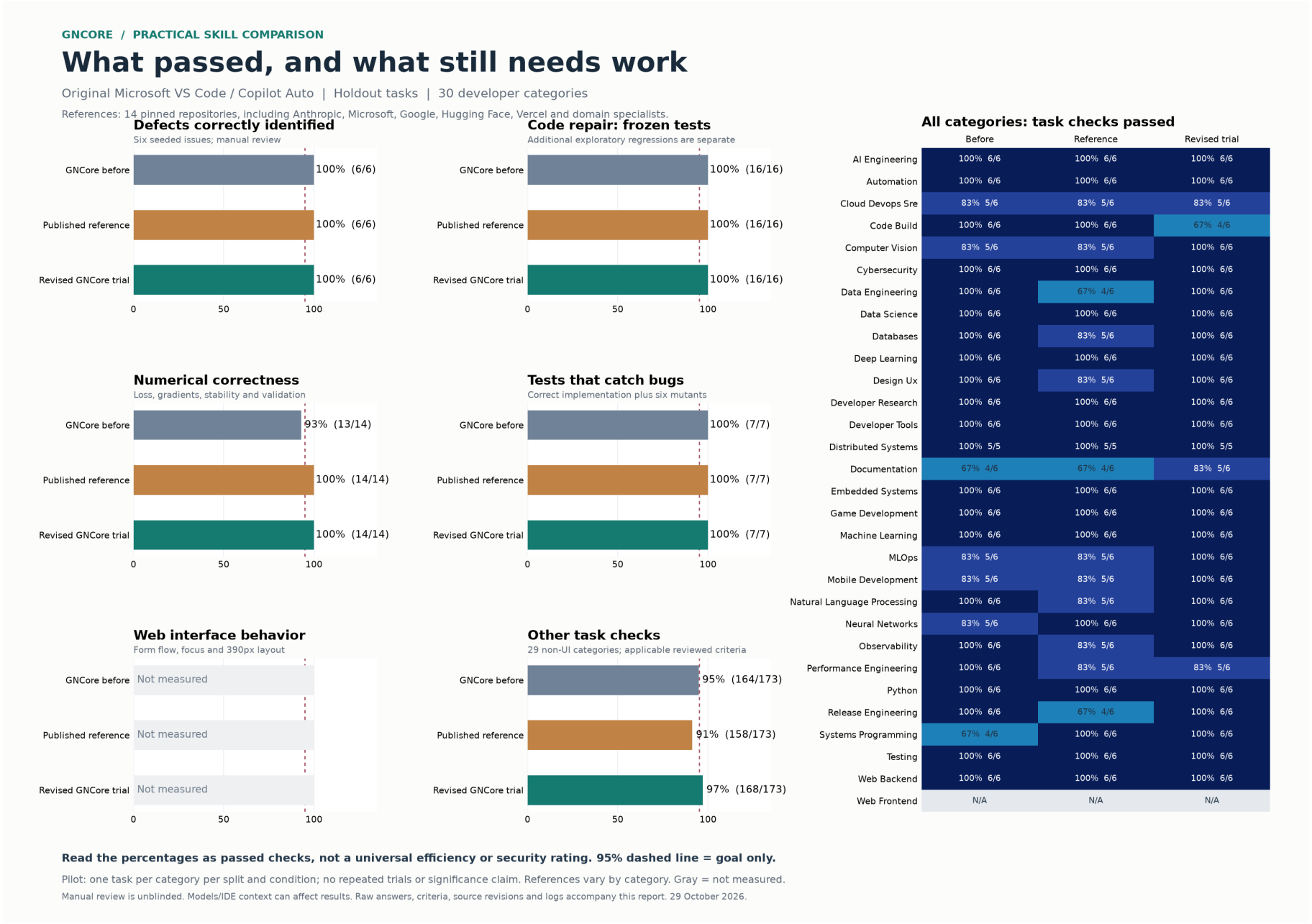
Package and installation checks

The complete Python3.12 package suite passed148 tests with one skip when temporary test files were outside the sync folder. A run inside the synced repository passed147, failed one and skipped one after Windows access-denied journal replacement. Both logs remain. Bounded atomic replacement retries and permanent-failure preservation tests were added; the competing process was not identified. This does not prove every synced-folder installation succeeds.

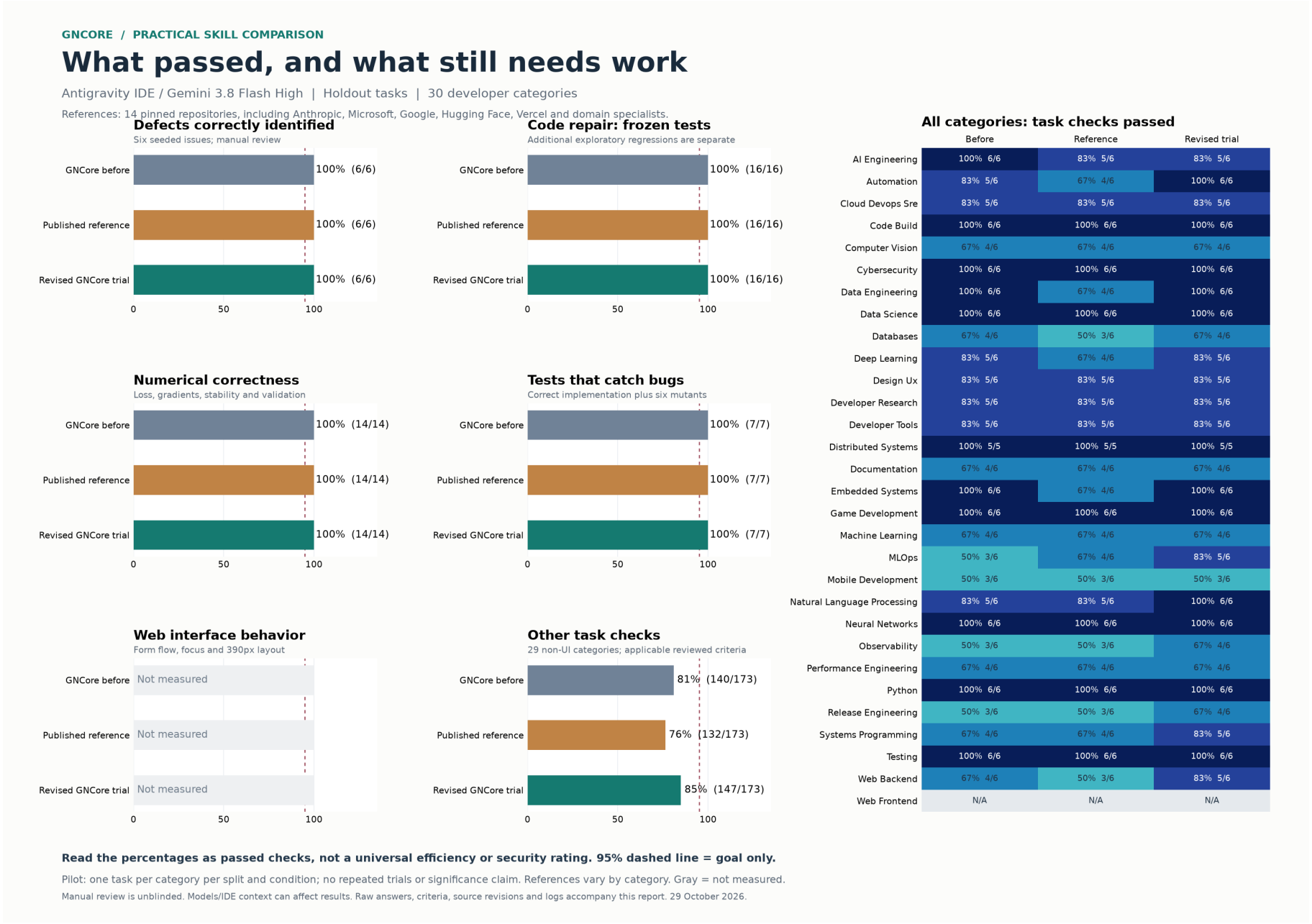
Release decision

Keep release publication on hold while reviewing the evidence and remaining failures. Prepare and verify package and website artifacts before asking the owner for GitHub, PyPI and website publication approval.

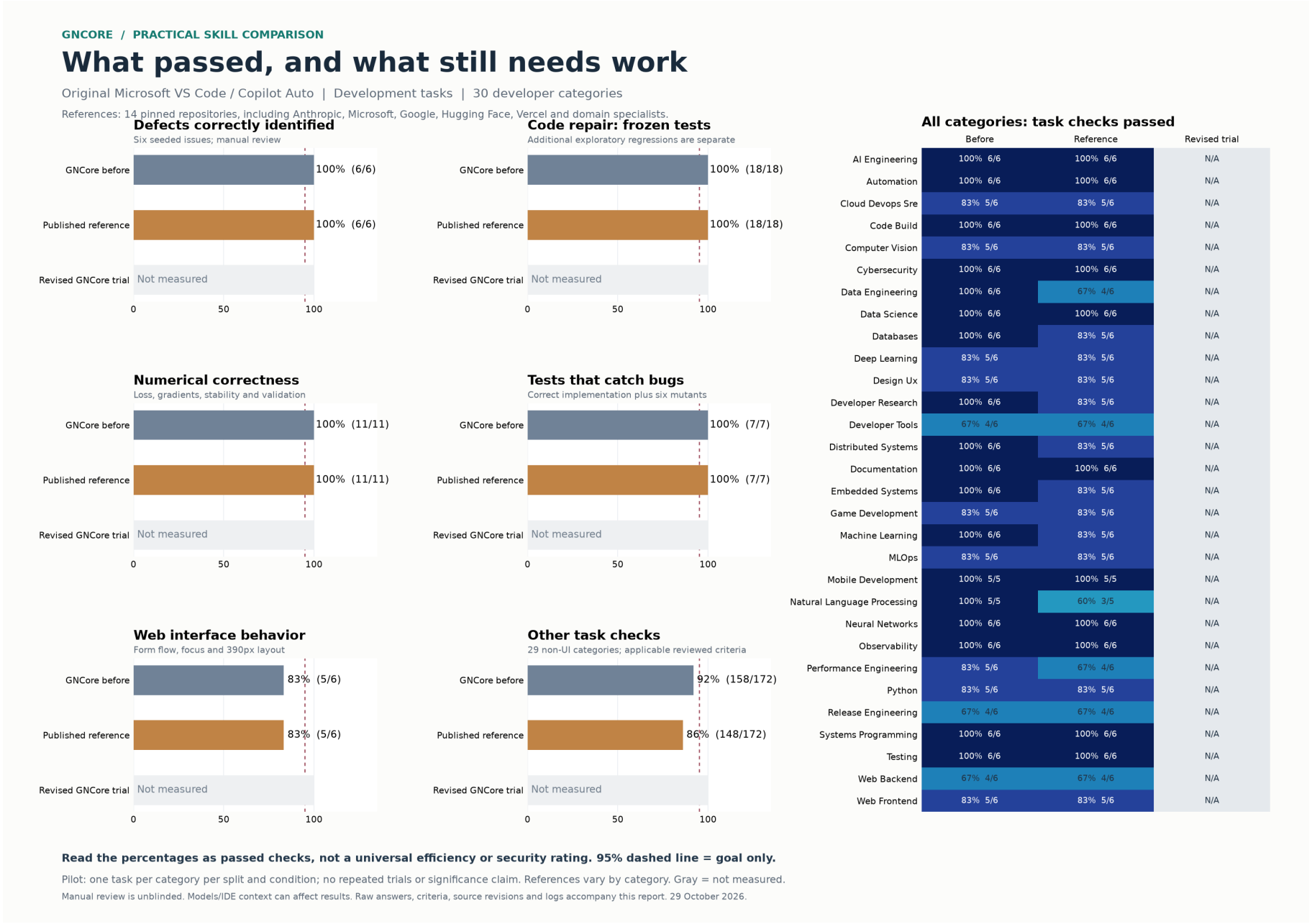
Vscode Holdout Landscape



Antigravity Holdout Landscape



Vscode Development Landscape



Antigravity Development Landscape

